

Storage Security Bucket Policy

Tasks-1. Create a new storage bucket in your chosen cloud provider (AWS S3, Google Cloud Storage, or Azure Blob Storage) named 'my-music-uploads' and upload any 2 MP3 files to it.

1. Create the bucket

Open the

Amazon S3 Console

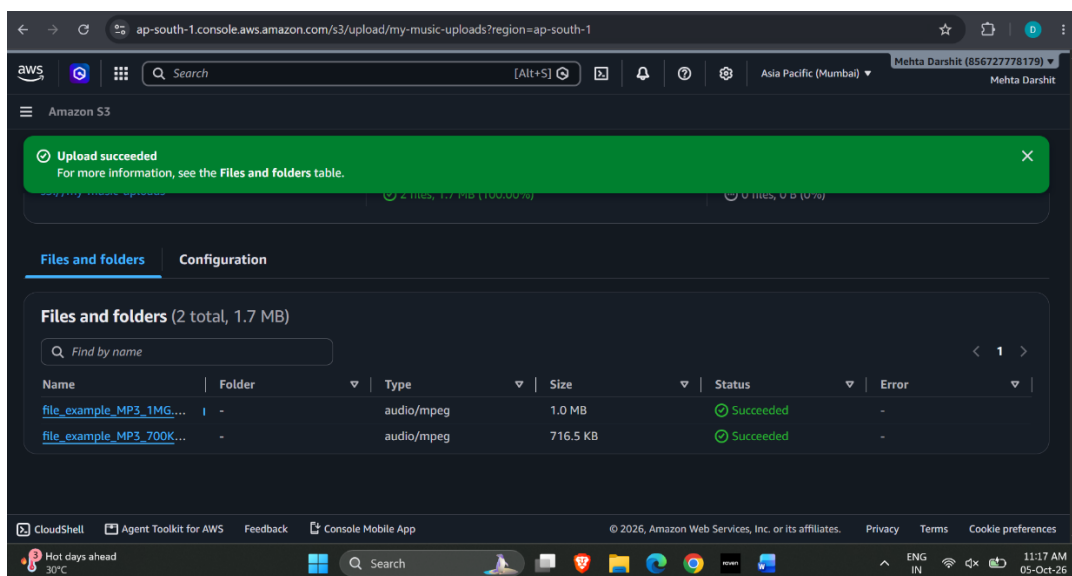
- Click Create bucket.
- Bucket name: my-music-uploads (if available).
- Region: Asia Pacific (Mumbai) — ap-south-1.
- Keep Block all public access enabled.
- Create the bucket.

2. Upload two MP3 files

Open the bucket → Upload → Add files. Choose two MP3 files you own or have permission to use, such as:

- music-01.mp3
- music-02.mp3

Click Upload and confirm that both objects appear in the Objects list.



Tasks-2.Apply a bucket policy to 'my-music-uploads' so only users with a specific email address (your own or a friend's) can download files from the bucket. Use the bucket's access management settings to add an IAM user or assign a permission for a specific email.

A. Create or identify the authorized IAM user

1. Open the IAM Console
2. Open Users and select an existing user or create one, for example music-downloader.
3. Record that user's ARN, which looks like:

arn:aws:iam::123456789012:user/music-downloader

The 12-digit number is a placeholder; use your own AWS account ID.

4. Make sure the user has the credentials and identity permissions needed to sign in or retrieve S3 objects securely.

B. Apply an access policy

In S3, open my-music-uploads → Permissions → Bucket policy → Edit.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowMusicUserFromApprovedIP",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:user/music-downloader"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-music-uploads/*",
      "Condition": {
```

```
"IpAddress": {
  "aws:SourceIp": "103.250.145.147/32"
}
},
{
  "Sid": "DenyDownloadsFromOtherIPs",
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3::my-music-uploads/*",
  "Condition": {
    "NotIpAddress": {
      "aws:SourceIp": "103.250.145.147/32"
    }
  }
},
{
  "Sid": "DenyDownloadsByOtherPrincipals",
  "Effect": "Deny",
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3::my-music-uploads/*",
  "Condition": {
    "ArnNotEquals": {
      "aws:PrincipalArn": [
```

```
"arn:aws:iam::123456789012:user/music-downloader",  
"arn:aws:iam::123456789012:role/YourAdminRole"  
]  
}  
}  
}  
]  
}
```

Tasks-3. Enable server-side encryption for all objects uploaded to 'my-music-uploads', ensuring that future uploads are encrypted automatically by the cloud provider.

🔗 **Open the bucket and select Properties.**

🔗 **Scroll to Default encryption and click Edit.**

🔗 **Select Server-side encryption with Amazon S3 managed keys (SSE-S3).**

🔗 **Save the changes.**

Tasks-4. Test your bucket's security by trying to access an uploaded MP3 file from a browser where you are not logged in — take a screenshot of the error or access denied message.

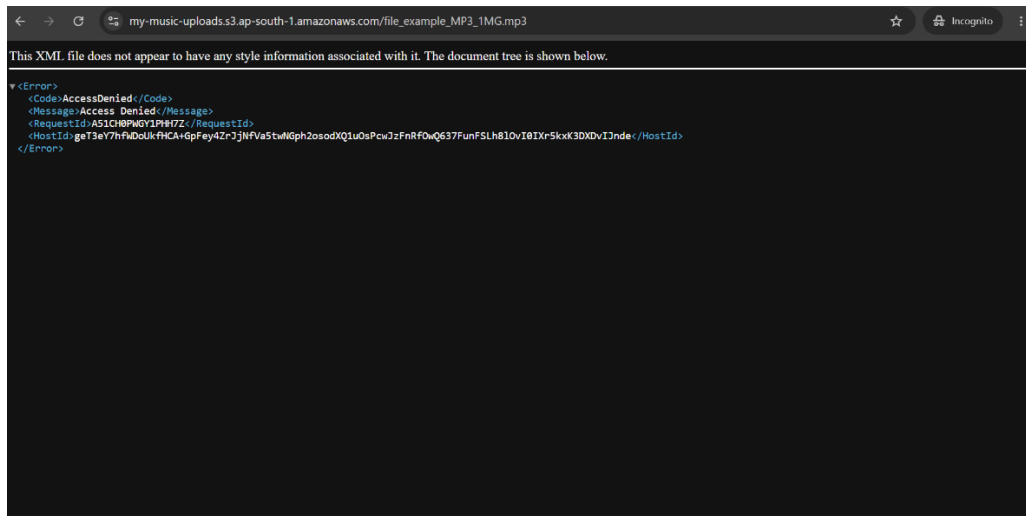
🔗 **Open one uploaded MP3 in the S3 Console and copy its Object URL.**

🔗 **Open a private/incognito browser window where you are not signed in to AWS.**

🔗 **Paste the object URL and press Enter.**

🔗 **With public access blocked and no presigned URL, you should receive an access-denied response, typically AccessDenied or an HTTP 403 error.**

🔗 **Take a screenshot showing the error.**



Tasks-5. Use an AI tool like ChatGPT or Copilot to generate a sample bucket policy that restricts access to your bucket by IP address, then modify and apply it to 'my-music-uploads'. Copy the AI-generated policy, review it for your cloud provider's syntax, and adjust the IP to your college's or your own network.

The sample policy in Task 2 already contains IP-based restrictions. To complete this task, replace its example IP address with your real public IP address or the public IP range supplied by your college's network administrator, then save the policy.